Report: Bayes Project

Mateus Auza Cruz and Belanov Bogning Tatchinda

29 mai 2025

Introduction

The goal of this project is to use a hierarchical Bayesian model to simulate data y using R and JAGS, and to evaluate the model's predictive performance. This approach allows us to incorporate both individual- and group-level variability in a principled probabilistic framework. In what follows, we build the model, derive the necessary components of Bayesian inference, and implement the full MCMC procedure to draw samples from the posterior distribution.

Question 1 : Model Derivation

We begin by formally specifying the probabilistic structure of the hierarchical model, followed by derivations of the likelihood and posterior distributions.

(a) Full Likelihood Function

To derive the full likelihood, we consider both the likelihood of the data and the prior distributions for the unknown parameters : $\beta = (\beta_0, \beta_1, \beta_2)$, the hyperparameter α, and the random effects $v_1, \ldots, v_G$ associated with each group (e.g., hospital).

Priors

— The regression coefficients β_j are assumed to be independent and normally distributed :

$$\beta_j \sim \mathcal{N}(0, 100) \quad \Rightarrow \quad p(\beta_0, \beta_1, \beta_2) \propto \prod_{j=0}^{2} \exp\left(-\frac{\beta_j^2}{200}\right)$$

— The hyperparameter α is assigned a Gamma prior :

$$\alpha \sim \text{Gamma}(a_0, b_0) \quad \Rightarrow \quad p(\alpha) \propto \alpha^{a_0 - 1} \exp(-\alpha b_0)$$

— The group-level random effects v_g are modeled as :

$$v_g \mid \alpha \sim \text{Gamma}(\alpha, \alpha) \quad \Rightarrow \quad p(v_1, \ldots, v_G \mid \alpha) \propto \prod_{g=1}^{G} v_g^{\alpha-1} \exp(-\alpha v_g)$$

Likelihood Let $\eta_i = \beta_0 + \beta_1 \cdot \text{age}_i + \beta_2 \cdot \text{chronic}_i$. The data y_i is assumed to follow a Poisson distribution :

$$y_i \mid v_{g(i)}, \beta \sim \text{Poisson}(v_{g(i)} \exp(\eta_i))$$

Therefore, the likelihood of the data is :

$$p(y_1, \ldots, y_n \mid \beta, v_1, \ldots, v_G) = \prod_{i=1}^{n} \frac{(v_{g(i)} \exp(\eta_i))^{y_i} \exp(-v_{g(i)} \exp(\eta_i))}{y_i!} \propto \prod_{i=1}^{n} (v_{g(i)} \exp(\eta_i))^{y_i} \exp(-v_{g(i)} \exp(\eta_i))$$

(b) Posterior Distribution

We now combine the priors and the likelihood to obtain the unnormalized joint posterior distribution :

$$p(\beta_0, \beta_1, \beta_2, v_1, \ldots, v_G, \alpha \mid y) \propto$$
$$\left[\prod_{i=1}^{n} (v_{g(i)} \exp(\eta_i))^{y_i} \exp(-v_{g(i)} \exp(\eta_i))\right] \cdot \left[\prod_{j=0}^{2} \exp\left(-\frac{\beta_j^2}{200}\right)\right] \cdot \alpha^{a_0-1} e^{-b_0 \alpha} \cdot \prod_{g=1}^{G} v_g^{\alpha-1} \exp(-\alpha v_g)$$

This expression does not simplify to a known analytical distribution. To perform inference, we will need to use sampling techniques such as Markov Chain Monte Carlo (MCMC) based on the full conditional distributions.

Question 2 : Full Conditional Distributions

We now derive the full conditional distributions needed to construct our Gibbs sampler and apply Metropolis steps when necessary.

(a) Regression Coefficients β

The conditional posterior for β is proportional to :

$$p(\beta \mid \cdot) \propto \prod_{i=1}^{n} (v_{g(i)} \exp(\eta_i))^{y_i} \exp(-v_{g(i)} \exp(\eta_i)) \cdot \prod_{j=0}^{2} \exp\left(-\frac{\beta_j^2}{200}\right)$$

Taking the log :

$$\log p(\beta \mid \cdot) \propto \sum_{i=1}^{n} \left(y_i \eta_i - v_{g(i)} \exp(\eta_i)\right) - \sum_{j=0}^{2} \frac{\beta_j^2}{200}$$

As this does not correspond to a standard distribution, we will use a Metropolis-Hastings step to sample from it.

(b) Random Effects v_g

For a specific group g, we collect the indices $i \in I_g$ where the observation belongs to group g. The conditional posterior becomes :

$$p(v_g \mid \cdot) \propto v_g^{\sum_{i \in I_g} y_i + \alpha - 1} \cdot \exp\left(-v_g \left(\sum_{i \in I_g} \exp(\eta_i) + \alpha\right)\right)$$

This is the kernel of a Gamma distribution :

$$v_g \mid \cdot \sim \text{Gamma}\left(\alpha + \sum_{i \in I_g} y_i,\ \alpha + \sum_{i \in I_g} \exp(\eta_i)\right)$$

As this is a known distribution, we will sample v_g using Gibbs steps.

(c) Hyperparameter α

The conditional posterior of α is given by :

$$p(\alpha \mid \cdot) \propto \alpha^{a_0 - 1} \exp(-b_0 \alpha) \cdot \prod_{g=1}^{G} v_g^{\alpha - 1} \exp(-\alpha v_g)$$

Taking the log :

$$\log p(\alpha \mid \cdot) \propto (a_0 - 1)\log(\alpha) - b_0 \alpha + (\alpha - 1)\sum_{g=1}^{G} \log(v_g) - \alpha \sum_{g=1}^{G} v_g$$

This distribution has no closed-form expression, so we will again use Metropolis-Hastings to sample α.

Question 3 : MCMC Implementation

To approximate the posterior distribution, we implement a Markov Chain Monte Carlo (MCMC) sampler in base R. Given the structure of our model, we use a Metropolis-within-Gibbs strategy.

Sampling Strategy

We alternate between :

- Updating β via a Metropolis-Hastings step.
- Updating v_g via Gibbs sampling from the Gamma full conditionals.

- Updating α via a Metropolis-Hastings step.

At each iteration t, we proceed as follows :

(i) Sample $\beta^{(t)}$ from $p(\beta \mid \alpha^{(t-1)}, v^{(t-1)}, \text{data})$ using Metropolis-Hastings.
(ii) Sample each $v_g^{(t)}$ from its full conditional using Gibbs.
(iii) Sample $\alpha^{(t)}$ from $p(\alpha \mid \beta^{(t)}, v^{(t)}, \text{data})$ using Metropolis-Hastings.

This procedure yields a sequence of samples whose stationary distribution approximates the joint posterior of all unknowns in the model.

Interpretations :

The results below summarize the posterior distributions of the parameters based on the MCMC output. These estimates provide insight into the model's key effects and variability.

To begin, we examine the traceplots and autocorrelation plots to assess the behavior of the MCMC chains for each parameter.

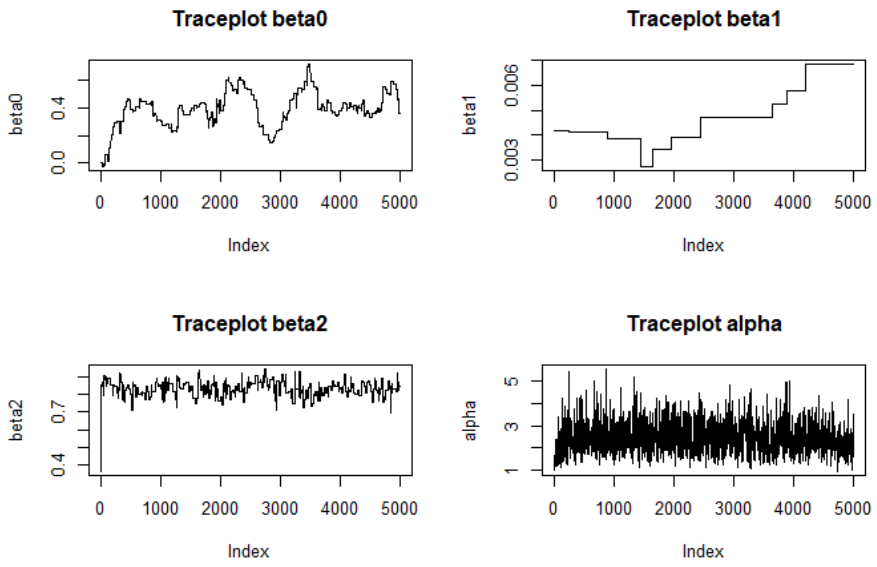

FIGURE 1 – Traceplots

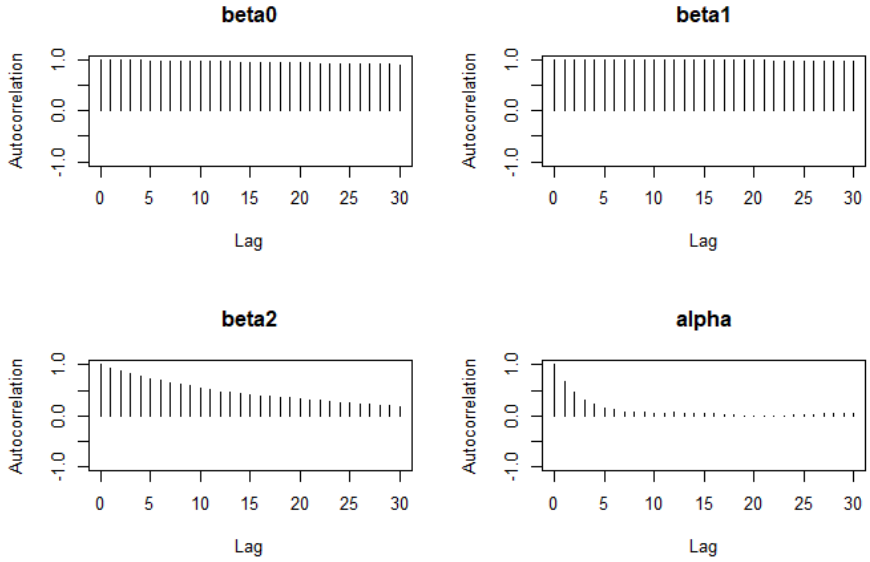

FIGURE 2 – Autocorrolation plots

From Figures 1 and 2 we can take the following conclusions :

Traceplot : `beta0`

— The chain moves slowly and shows strong autocorrelation.

— Poor mixing. The chain does not explore the parameter space efficiently.

Traceplot : `beta1`

— The chain is essentially flat for long stretches, with only occasional jumps.

— Extremely poor mixing. This parameter is not moving much, leading to inefficient sampling.

Traceplot : `beta2`

— Better than `beta0` and `beta1`, but still shows signs of autocorrelation.

— Moderately poor mixing. The chain is exploring the parameter space, but slowly.

Traceplot : `alpha`

— The chain appears to jump around freely across the range.

— Good mixing — the parameter space is well explored, with low autocorrelation.

— The sampler is effectively drawing from the posterior for `alpha`. No immediate action is required.

Acessing convergence :

To assess convergence using only a Markov Chain, we use the Geweke diagnostic (the R statistic requires multiple MCMC's which we do not have).

Next, we turn to the Geweke diagnostic outputs to formally evaluate convergence.

```
Fraction in 1st window = 0.1
Fraction in 2nd window = 0.5

  beta0   beta1   beta2   alpha
 1.1796 -1.2727 -0.8020 -0.2167
```

FIGURE 3 – Geweke diagnostic

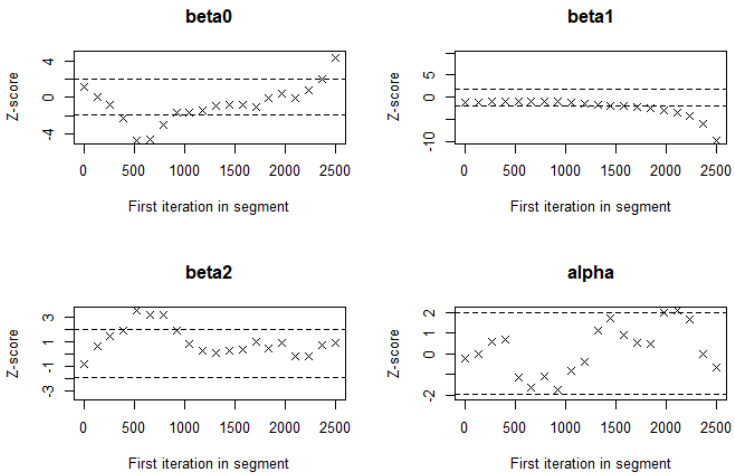

FIGURE 4 – Geweke plot

From Figures 3 and 4, we observe the following :

— None of the values from these chains have absolute Geweke statistics greater, in absolute value, than 2.
— According to the Geweke diagnostic, there is no strong evidence against convergence.
— The Geweke plots themselves are inconclusive, but do not indicate severe non-convergence.

Overall, these diagnostics suggest that while some parameters mix poorly, the chain appears to have converged reasonably well.

Question 4 : Posterior Analysis

To improve the reliability of the posterior analysis, we discard the first 1000 iterations as burn-in. This step ensures that the resulting summaries are not influenced by the initial values of the parameters.

Now, let us evaluate the posterior credible intervals and numerical summaries of the parameters, followed by graphical insights into the group-level effects.

(a) Provide numerical summaries and credible intervals for α, β. Provide graphical summaries for v_g

We begin by examining the numerical posterior summaries of the fixed effects and the dispersion parameter.

TABLE 1 – Posterior Summaries for β and α

	Mean	**SD**	**Median**	**Min**	**Max**	**2.5%**	**97.5%**
Beta0	0.4089	0.1164	0.3994	0.1448	0.7170	0.1886	0.6036
Beta1	0.0049	0.0012	0.0047	0.0028	0.0069	0.0028	0.0069
Beta2	0.8338	0.0411	0.8358	0.6921	0.9432	0.7491	0.9142
Alpha	2.4536	0.6502	2.3729	0.9289	5.1817	1.4116	3.9007

From Table 1, we can make the following interpretations :

Beta0 The intercept (mean 0.41) represents the baseline log-mean outcome when all covariates are zero. This value may have limited interpretability depending on covariate scaling.

Beta1 This coefficient has a small magnitude (mean 0.005), but the 95% credible interval does not include zero, suggesting a consistent, though modest, positive effect of this covariate on the log outcome.

Beta2 The estimate (mean 0.83) suggests a strong and well-supported positive effect, with the credible interval entirely above zero, indicating substantial influence on the log-mean response.

Alpha The hyperparameter α (mean 2.45) governs the dispersion of hospital-specific effects. This indicates moderate overdispersion across hospitals, reflecting heterogeneity beyond the Poisson model.

Next, we turn our attention to the graphical summaries for the hospital-level effects.

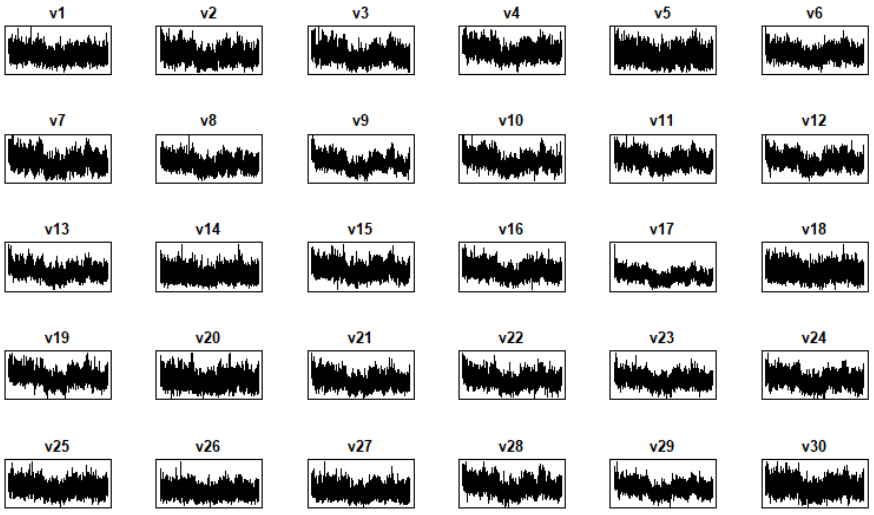

FIGURE 5 – Graphical summaries for v_g

From Figure 5, we observe that the hospital-specific random effectsv_g exhibit good mixing. This indicates that the parameter space has been well explored, with low autocorrelation across chains.

Together, these numerical and graphical summaries provide a comprehensive picture of the fixed and group-level effects in the model.

(b) Evaluate the posterior predictive performance of the model

The goal of this analysis is to assess the predictive performance of the posterior distribution. To do this, we used the posterior samples obtained in Question 3 and compared the density of the posterior predictive distribution with the observed values of y, as shown in Figure 6.

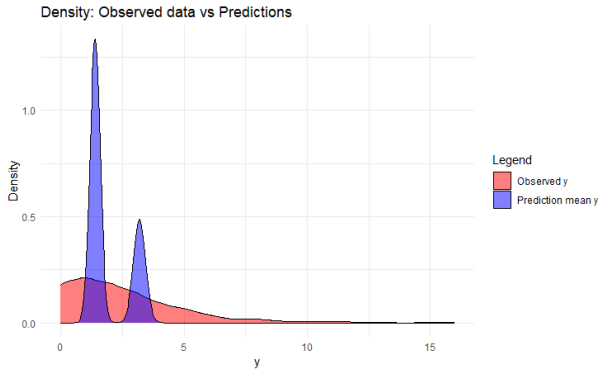

FIGURE 6 – Comparison of posterior predictive distribution and observed data

Based on this density plot, we can draw the following conclusions regarding the model's predictive performance :

— The model predicts the mean of the observed y values quite accurately.
— The model underestimates the variability present in the observed data.
— The posterior predictive distribution has two bell shapes, which suggests the possibility of unmodeled heterogeneity.

From these observations, we can conclude that the model is capable of generating samples that are close to the mean of the observed values, but it struggles to capture outliers and extreme values. Therefore, it can be considered a reasonably good model for central tendency, though limited in capturing the full spread of the data.

Given the potential for heterogeneity across hospitals, it may seem reasonable to incorporate random effects to account for this variation and improve model fit. This motivates the use of a hierarchical model, which explicitly models hospital-specific effects through a random structure. However, model comparison using the Deviance Information Criterion (DIC), as shown in Table 2, reveals that the simple model actually yields a lower DIC (3132.7 vs. 3792.1). Despite the hierarchical model being more flexible and having a higher number of effective parameters (29.8 vs. 3), its fit to the data is not sufficient to compensate for the increased complexity. Thus, according to DIC, the simpler model is preferred.

	DIC	**# of effective parameters**
Simple model	3132.7	3
Hierarchical model	3792.1	29.8

TABLE 2 – Model comparison based on DIC and effective number of parameters.

This conclusion is further supported by the predictive comparison in Figure 7, where the hierarchical model exhibits greater predictive uncertainty (larger standard deviation)- while offering no clear improvement in predictive accuracy.

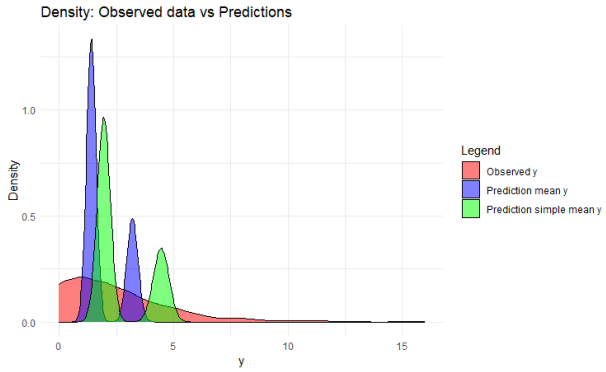

FIGURE 7 – Visual comparison between the simple and hierarchical models

Question 5 : JAGS Comparison

(a) Use JAGS to simulate from the posterior

We implemented the hierarchical model in JAGS using the *rjags* package in R. The code used to specify the model, define priors, and run the MCMC sampler is provided in the accompanying R script.

(b) Compare the results from JAGS and the base R implementation

After fitting the model using both base R (with a Metropolis-within-Gibbs sampler) and JAGS, we compare the posterior summaries of the parameters. Tables 3 and 4 report the posterior means and standard deviations for both implementations. Additionally, Figures 8, 9 and 10 provide a visual comparison of the posterior distributions for the parameters obtained from the two methods.

	Parameter	**Mean (R)**	**SD (R)**	**Mean (JAGS)**	**SD (JAGS)**
beta0	beta0	0.4088500	0.1163983	0.1870402	0.1380004
beta1	beta1	0.0048610	0.0012157	0.0077106	0.0021050
beta2	beta2	0.8338086	0.0410788	0.8209330	0.0429143

TABLE 3 – Comparison of Posterior Summaries for β Parameters (R vs. JAGS)

Parameter	**Mean (R)**	**SD (R)**	**Mean (JAGS)**	**SD (JAGS)**
alpha	2.453556	0.6502305	2.891386	0.7554139

TABLE 4 – Comparison of Posterior Summaries for α (R vs. JAGS)

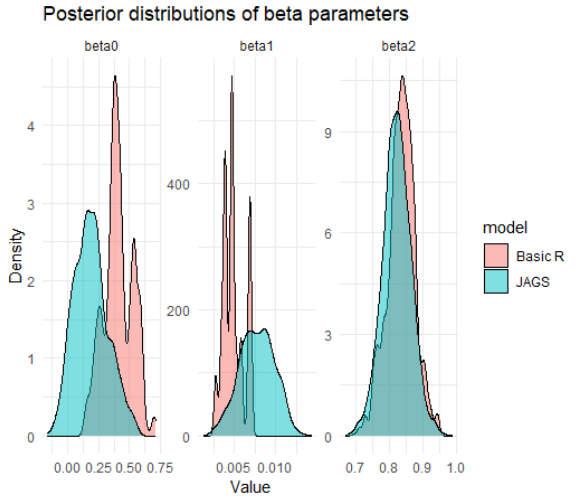

FIGURE 8 – Posterior comparison for β coefficients

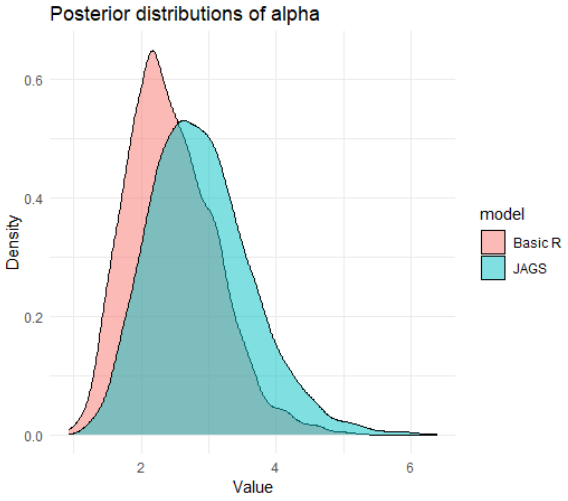

FIGURE 9 – Posterior comparison for α

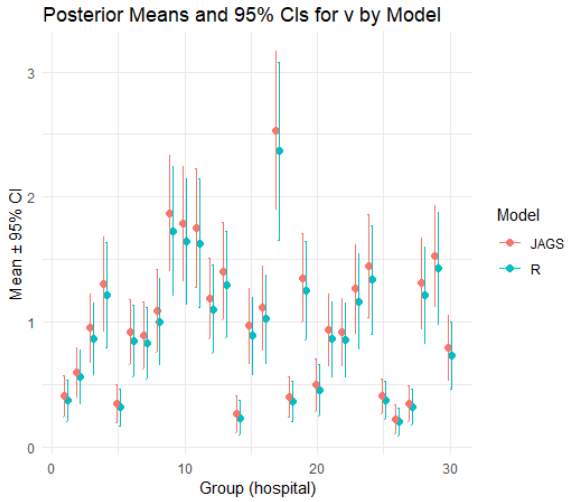

FIGURE 10 – Comparison of posterior means for hospital-level effects

From these results, we observe that :

— JAGS and base R yield similar posterior trends, but with slight differences in location and spread.

— The group-level (hospital) random effects show similar patterns in both methods, suggesting both approaches capture the same hierarchical structure.

Overall, while some parameter estimates differ marginally, both implementations support the same substantive conclusions about the data.

Question 6 : Robustness to Prior Choice

Replace the prior $V_g \sim Gamma(\alpha, \alpha)$ *with a log-normal specification :*

$$\log V_g \sim \mathcal{N}(0, \tau^{-1}), \quad \tau \sim Gamma(a_0, b_0) \quad with\ a_0 = b_0 = 0.01$$

Update your JAGS code accordingly and compare the resulting posterior distributions.

After updating our JAGS model (calling it JAGS2), using jags.model, we obtain the following results :

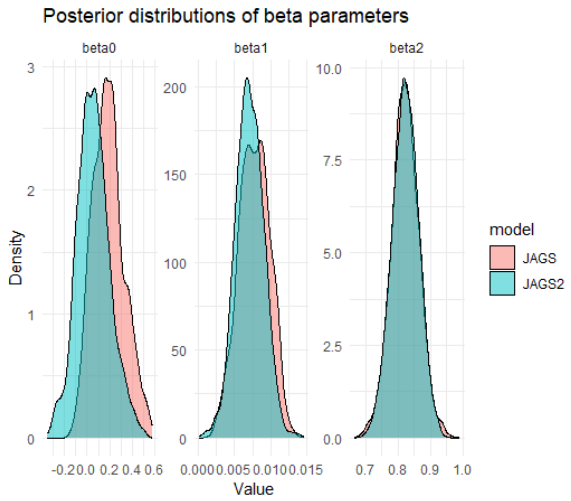

FIGURE 11 – Posterior comparison for β coefficients

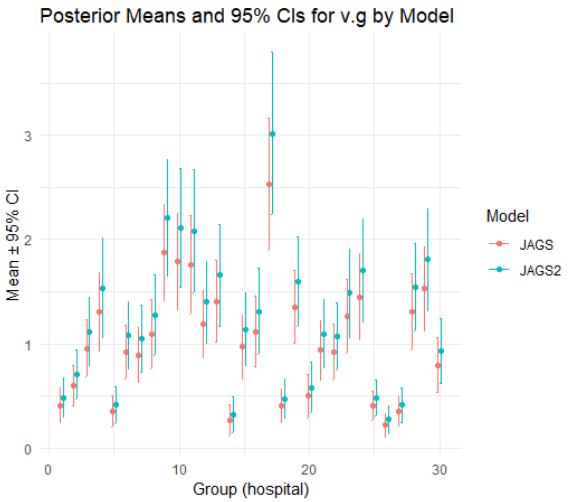

FIGURE 12 – Comparison of posterior means for hospital-level effects

From these results, we can conclude the following :

— The density curves for the fixed-effect parameters β are very similar under both models, suggesting that these parameters are **robust to prior choices**.

— The hospital-specific random effects v_g, however, **show sensitivity to the choice of prior**, highlighting the influence of prior assumptions in hierarchical modeling.

Thus, we observe that **the overall model is not robust to prior specification**, particularly due to the hierarchical structure governing v_g.

Conclusion

To summarize, in **Question 4.b**, we found that although the model could accurately predict values near the mean, it performed **poorly for outliers and extreme observations**, indicating **modest predictive performance**.

Furthermore, as seen in **Question 6**, the model is **highly sensitive to the prior on** v_g. This is a notable limitation, as it undermines the model's robustness and reliability.

To address these concerns, a simpler model—as proposed in Question 4.b—could be a viable alternative. This non-hierarchical model is **not sensitive to the prior for** v_g, and it demonstrated a **lower DIC**, indicating **better predictive performance** overall.

Appendix :

Software Code :

```r
#########################################################################
###############Pre processing the data ###################################
#########################################################################

#Libraries
library(coda)
library(knitr)
library(ggplot2)
library(rjags)
library(tidyr)
library(dplyr)

set.seed(219)
# Load the dataset
data = read.table("HospitalVisits.txt", header = TRUE)

# Prepare data
n = nrow(data)
group = data$hospital
g = as.numeric(as.factor(group))
G = length(unique(g)) #Or max(g)-same thing
y = data$y
age = data$age
chronic = data$chronic
X = cbind(1, age, chronic)  # Design matrix

# Initial values
beta = c(0, 0, 0)
v = rep(1, G)
alpha = 1

# Prior parameters
a.0 = 0.01
b.0 = 0.01

#################################################
#################Question 3######################
#################################################

#Variances of the proposal
sd.beta=0.5
sd.alpha=0.5

# Monte Carlo Markov Chain setup
n_iter = 5000
samples = matrix(0, nrow=n_iter, ncol=length(beta) + G + 1)
```

```r
colnames(samples) = c(paste0("beta", 0:2), paste0("v", 1:G), "alpha")

# Log-posterior for beta
log.post.beta = function(beta, v, alpha) {
  eta = X %*% beta
  mu = v[g] * exp(eta)
  lik = sum(y * eta - mu)
  lp.b = sum(-beta^2 / 200)
  return(lik + lp.b)
}

# Log-posterior for alpha
log.post.alpha = function(alpha, v) {
  if (alpha <= 0) return(-Inf)
  G = length(v)
  lp.a = (a.0 - 1) * log(alpha) - b.0 * alpha
  lp.a = lp.a + sum((alpha - 1) * log(v) - alpha * v)+ G*(alpha*log(alpha) -
      lgamma(alpha))
  return(lp.a)
}

# MCMC sampler
for (iter in 1:n_iter) {

  # 1. Update beta via Metropolis
  for (j in 1:3) {
    beta.prop = beta
    beta.prop[j] = rnorm(1, beta[j], sd = sd.beta)
    log.r = log.post.beta(beta.prop, v, alpha) - log.post.beta(beta, v, alpha)
    if (log(runif(1)) < log.r) beta = beta.prop
  }

  # 2. Update v via Gibbs
  for (h in 1:G) {
    idx = which(g == h)
    eta_h = X[idx, ] %*% beta
    shape = alpha + sum(y[idx])
    rate = alpha + sum(exp(eta_h))
    v[h] = rgamma(1, shape, rate)
  }

  # 3. Update alpha via Metropolis-Hastings on log-scale!
  log.alpha.prop = rnorm(1, log(alpha), sd = sd.alpha)
  alpha.prop = exp(log.alpha.prop)

  log.r = log.post.alpha(alpha.prop, v) - log.post.alpha(alpha, v) - log.alpha.
      prop + log(alpha)
  if (log(runif(1)) < log.r) alpha = alpha.prop

  # Save samples
  samples[iter, ] = c(beta, v, alpha)
```

```
}

# Basic diagnostics
par(mfrow=c(2,2))
plot(samples[, "beta0"], type="l", main="Traceplot beta0",ylab="beta0")
plot(samples[, "beta1"], type="l", main="Traceplot beta1",ylab="beta1")
plot(samples[, "beta2"], type="l", main="Traceplot beta2",ylab="beta2")
plot(samples[, "alpha"], type="l", main="Traceplot alpha",ylab="alpha")


cat("Posterior means:\n")
print(colMeans(samples))

########Computing the Geweke statistic for the chains alpha and beta

convergence_mcmc = as.mcmc(samples[, c("beta0", "beta1", "beta2","alpha")])
geweke.diag(convergence_mcmc)

##Plots of the convergence
geweke.plot(convergence_mcmc)

###Autocorrolation plots
autocorr.plot(convergence_mcmc)

# --------------------------------
# EXERCISE 4: Posterior Analysis
# --------------------------------



# (a) Numerical summaries and credible intervals for beta and alpha
burn.in = 1000  # Adjust if needed

# Extract posterior samples after burn-in
post.samples = samples[(burn.in + 1):n_iter, ]

# Separate beta and alpha
post.beta = post.samples[, c("beta0", "beta1", "beta2")]
post.alpha = post.samples[, "alpha"]

# Summary function
summary.stats = function(samples) {
  c(
    Mean    = round(mean(samples), 4),
    SD      = round(sd(samples), 4),
    Median  = round(median(samples), 4),
    Min     = round(min(samples), 4),
    Max     = round(max(samples), 4),
    `2.5%`  = round(quantile(samples, 0.025), 4),
    `97.5%` = round(quantile(samples, 0.975), 4)
  )
```

```r
}

# Combine summaries
Matrix = rbind(
  summary.stats(post.beta[, 1]),
  summary.stats(post.beta[, 2]),
  summary.stats(post.beta[, 3]),
  summary.stats(post.alpha)
)

rownames(Matrix) = c("Beta0", "Beta1", "Beta2", "Alpha")

# Print nicely
kable(Matrix, format = "markdown", caption = "Posterior␣Summaries␣for␣  ␣and␣  "
    )

# ------------------------------
# (b) Graphical summaries for v_g
# ------------------------------

# Extract v_g samples (columns 4 to 4 + G - 1)
post.v = post.samples[, paste0("v", 1:G)]

# Traceplots for v_g (30 hospitals)
par(mfrow = c(5, 6), mar = c(2, 2, 2, 1))
for (g in 1:G) {
  plot(post.v[, g], type = "l", main = paste0("v", g), xaxt = 'n', yaxt = 'n')
}

# (b) Evaluate the posterior predictive performance of the fitted model

# Sample S posterior draws for prediction
S = 1000
sample.indices = sample(1:nrow(post.beta), S)

# Matrix to store synthetic outcomes
y.simul = matrix(0, nrow = S, ncol = length(y))

# Generate posterior predictive draws
for (s in 1:S) {
  beta.s = post.beta[sample.indices[s], ]
  v.s = post.v[sample.indices[s], ][g]  # g = hospital ID index from earlier
  eta.s = beta.s[1] + beta.s[2] * age + beta.s[3] * chronic
  y.simul[s, ] = rpois(length(y), lambda = v.s * exp(eta.s))
}

########################################
######################Visualisation of the posterior density compared with the
    true model#####################################################
########################################

```

```r
# Compute average prediction per observation
y.pred.mean = colMeans(y.simul)

# Create a combined data frame
dens.df = bind_rows(
  data.frame(value = y, type = "Observed y"),
  data.frame(value = y.pred.mean, type = "Prediction mean y")
)

# Plot with ggplot2
ggplot(dens.df, aes(x = value, fill = type)) +
  geom_density(alpha = 0.5) +
  theme_minimal() +
  labs(
    title = "Density: Observed data vs Predictions",
    x = "y",
    y = "Density",
    fill = "Legend"
  ) +
  scale_fill_manual(values = c("Observed y" = "red", "Prediction mean y" = "blue"))


### Numerical comparison
# Compare mean
cat("Observed mean of y:", mean(y), "\n")
cat("Posterior predictive mean of y:", mean(apply(y.simul, 1, mean)), "\n")

# Compare variance
cat("Observed SD of y:", sd(y), "\n")
cat("Average posterior predictive SD:", mean(apply(y.simul, 1, sd)), "\n")

#From these numerical and graphical summaries we can see that the model tends to locate quite accurately the mean of the observed distribution but underestimates the variability of the observed y.


##########################################################
################Computing the DIC#####################
##########################################################
#Now we compare the hierachical model (with a hyperparameter v.g) and a non hierarchical model


###DIC for the hierarchical model ###
deviance_samples = numeric(nrow(post.beta))
for (s in 1:nrow(post.beta)) {
  beta_s = post.beta[s, ]
  v_s = post.v[s, ]
  mu_s = exp(X %*% beta_s)*v_s[group]
  deviance_samples[s] <- -2 * sum(dpois(y, mu_s, log = TRUE))
```

```r
}

# DIC
D_bar = mean(deviance_samples)
beta_mean = colMeans(post.beta)
v_mean = colMeans(post.v)
mu_mean = exp(X %*% beta_mean)*v_mean[group]
D_hat = -2 * sum(dpois(y, mu_mean, log = TRUE))
pD = D_bar - D_hat
DIC = D_bar + pD

# Mod le Poisson sans effets al atoires
beta_mean_simple = colMeans(post.beta)  # Moyenne a posteriori des beta
mu_mean_simple = exp(X %*% beta_mean_simple)
deviance_simple = -2 * sum(dpois(y, mu_mean_simple, log = TRUE))
pD_simple = 3  # Nombre de param tres (beta0, beta1, beta2)
DIC_simple = deviance_simple + pD_simple

#-----------------------------------------
# Comparison of the two models
#-----------------------------------------
# Results for the hierarchical model
cat("DIC (hi rarchique) =", round(DIC, 1), "\npD =", round(pD, 1))
# Results for the non hierarchical model
cat("DIC (simple) =", round(DIC_simple, 1), "\npD =", pD_simple)


##############################Simple model
# Sample S posterior draws for prediction
S = 1000
sample.indices = sample(1:nrow(post.beta), S)

# Matrix to store synthetic outcomes
y.simul.simple = matrix(0, nrow = S, ncol = length(y))

# Generate posterior predictive draws
for (s in 1:S) {
  beta.s = post.beta[sample.indices[s], ]
  eta.s = beta.s[1] + beta.s[2] * age + beta.s[3] * chronic
  y.simul.simple[s, ] = rpois(length(y), lambda =exp(eta.s))
}

# Compute average prediction per observation
y.pred.simple.mean = colMeans(y.simul.simple)

# Create a combined data frame
dens.df = bind_rows(
  data.frame(value = y, type = "Observed y"),
  data.frame(value = y.pred.mean, type = "Prediction mean y"),
  data.frame(value= y.pred.simple.mean, type="Prediction simple mean y")
)
```

```r

# Plot with ggplot2
ggplot(dens.df, aes(x = value, fill = type)) +
  geom_density(alpha = 0.5) +
  theme_minimal() +
  labs(
    title = "Density: Observed data vs Predictions",
    x = "y",
    y = "Density",
    fill = "Legend"
  ) +
  scale_fill_manual(values = c("Observed y" = "red", "Prediction mean y" = "blue", "Prediction simple mean y"="green"))



################################################
## Exercice 5 ##################################
################################################

#(a) Do the same as exercice 3 but using JAGS (Just Another Gibs Sampler)

#Putting the model into a string model
model.string = "
model {
  for (i in 1:N) {
    y[i] ~ dpois(lambda[i])
    log(lambda[i]) <- log(v[hospital[i]]) + beta0 + beta1 * age[i] + beta2 * chronic[i]
  }

  for (g in 1:G) {
    v[g] ~ dgamma(alpha, alpha)
  }

  # Priors
  beta0 ~ dnorm(0, 10)
  beta1 ~ dnorm(0, 10)
  beta2 ~ dnorm(0, 10)
  alpha ~ dgamma(a0, b0)
}
"

#Using JAGS
data.jags = list(
  y = y,
  age = age,
  chronic = chronic,
  hospital = data$hospital,
  N = nrow(data),
  G = G,
```

```
  a0 = a.0,
  b0 = b.0
)
initial.values = function() {
  list(
    beta0 = 0, beta1 = 0, beta2 = 0,
    alpha = 1,
    v = rep(1, data.jags$G)
  )
}
parameters=c("beta0","beta1","beta2","alpha","v")

#Model that stores everything-Interface between R and JAGS

model = jags.model(textConnection(model.string),
                   data = data.jags,
                   inits = initial.values,
                   n.chains = 1,
                   n.adapt = 1000)

#We update the model such that the model forgets the iterations before the burn-in
update(model,burn.in)

#We take the samples
samples.jags=coda.samples(model, variable.names = parameters,n_iter)

#Storing this in a matrix

samples.mat=as.matrix(samples.jags)

post.beta.jags = samples.mat[, c("beta0", "beta1", "beta2")]
post.alpha.jags = samples.mat[, "alpha"]
post.v.jags = samples.mat[, grep("^v\\[", colnames(samples.mat))]

#(b) Compare the posterior summaries obtained in 3 ######################

###Post.beta vs Post.beta.jags #################

comparison.beta = data.frame(
  Parameter = c("beta0", "beta1", "beta2"),
  Mean.R = colMeans(post.beta),
  SD.R   = apply(post.beta, 2, sd),
  Mean.JAGS   = colMeans(post.beta.jags),
  SD.JAGS     = apply(post.beta.jags, 2, sd)
)
kable(comparison.beta, format = "markdown", caption = "Comparison␣between␣the␣beta␣parameters␣in␣R␣and␣JAGS")

###Alpha ########################################

```

```r
comparison.alpha=data.frame(
  Parameter = "alpha",
  Mean.R = mean(post.alpha),
  SD.R   = sd(post.alpha),
  Mean.JAGS    = mean(post.alpha.jags),
  SD.JAGS      = sd(post.alpha.jags)
)
kable(comparison.alpha, format = "markdown", caption = "Comparison between the alpha in R and JAGS")


### Comparison v ################################

comparison.v= data.frame(
  Mean.R = colMeans(post.v),
  SD.R   = apply(post.v, 2, sd),
  Mean.JAGS    = colMeans(post.v.jags),
  SD.JAGS      = apply(post.v.jags, 2, sd)
)
kable(comparison.v, format = "markdown", caption = "Comparison between the v parameters in R and JAGS")

############################
#####Visual representation #
############################

###Beta
dif.beta = rbind(
  data.frame(value = as.vector(post.beta),
             parameter = rep(c("beta0", "beta1", "beta2"), each = nrow(post.beta)),
             model = "Basic R"),
  data.frame(value = as.vector(post.beta.jags),
             parameter = rep(c("beta0", "beta1", "beta2"), each = nrow(post.beta.jags)),
             model = "JAGS")
)

ggplot(dif.beta, aes(x = value, fill = model)) +
  geom_density(alpha = 0.5) +
  facet_wrap(~parameter, scales = "free") +
  theme_minimal() +
  labs(title = "Posterior distributions of beta parameters",
       x = "Value", y = "Density")

###Alpha ##############################################3

dif.alpha = rbind(
  data.frame(value = as.vector(post.alpha),
             parameter = "alpha",
             model = "Basic R"),
```

```r
  data.frame(value = as.vector(post.alpha.jags),
             parameter = "alpha",
             model = "JAGS")
)


ggplot(dif.alpha, aes(x = value, fill = model)) +
  geom_density(alpha = 0.5) +
  theme_minimal() +
  labs(title = "Posterior distributions of alpha",
       x = "Value", y = "Density")


###v ##########################################


# Add group index
comparison.v = comparison.v %>%
  mutate(Group = 1:n())


v.small = bind_rows(
  comparison.v %>%
    transmute(
      Group,
      Model = "R",
      Mean = Mean.R,
      Lower = Mean.R - 1.96 * SD.R,
      Upper = Mean.R + 1.96 * SD.R
    ),
  comparison.v %>%
    transmute(
      Group,
      Model = "JAGS",
      Mean = Mean.JAGS,
      Lower = Mean.JAGS - 1.96 * SD.JAGS,
      Upper = Mean.JAGS + 1.96 * SD.JAGS
    )
)

ggplot(v.small, aes(x = Group, y = Mean, color = Model)) +
  geom_point(position = position_dodge(width = 0.5), size = 2) +
  geom_errorbar(aes(ymin = Lower, ymax = Upper),
                width = 0.2,
                position = position_dodge(width = 0.5)) +
  theme_minimal() +
  labs(title = "Posterior Means and 95% CIs for v by Model",
       x = "Group (hospital)",
       y = "Mean   95% CI")

############################################################
```

```
################ Exercice 6- Robustness to the prior choice
############################################################

#Replacing the prior

model.string2 = "
model␣{
␣␣for␣(i␣in␣1:N)␣{
␣␣␣␣y[i]␣~␣dpois(lambda[i])
␣␣␣␣log(lambda[i])␣<-␣log(v[hospital[i]])␣+␣beta0␣+␣beta1␣*␣age[i]␣+␣beta2␣*␣
    chronic[i]
␣␣}

␣␣for␣(g␣in␣1:G)␣{
␣␣␣␣log.v[g]␣~␣dnorm(0,␣tau)
␣␣␣␣v[g]␣<-␣exp(log.v[g])
␣␣}

␣␣#␣Priors
␣␣beta0␣~␣dnorm(0,␣10)
␣␣beta1␣~␣dnorm(0,␣10)
␣␣beta2␣~␣dnorm(0,␣10)
␣␣tau␣~␣dgamma(a0,␣b0)
}
"
initial.values2 = function() {
  list(beta0 = 0,beta1 = 0,beta2 = 0,tau = 1,log.v = rep(0, G))}

parameters2=c("beta0","beta1","beta2","tau","v")

#Model that stores everything-Interface between R and JAGS

model2 = jags.model(textConnection(model.string2),
                    data = data.jags,
                    inits = initial.values2,
                    n.chains = 1,
                    n.adapt = 1000)

update(model2,burn.in)

#We take the samples
samples.jags2=coda.samples(model2, variable.names = parameters2,n_iter)

#Storing this in a matrix
samples.mat2=as.matrix(samples.jags2)

post.beta.jags2 = samples.mat2[, c("beta0", "beta1", "beta2")]
post.tau.jags = samples.mat2[, "tau"]
samples.mat2
post.v.jags2 = samples.mat2[, grep("^v\\[", colnames(samples.mat2))]

```

```r
##Differences between the two JAGS models

###Post.beta.jags vs Post.beta.jags2

comparison.beta.jags = data.frame(
  Parameter = c("beta0", "beta1", "beta2"),
  Mean.JAGS = colMeans(post.beta.jags),
  SD.JAGS   = apply(post.beta.jags, 2, sd),
  Mean.JAGS2   = colMeans(post.beta.jags2),
  SD.JAGS2     = apply(post.beta.jags2, 2, sd)
)
kable(comparison.beta.jags, format = "markdown", caption = "Comparison between the v parameters in R and JAGS")


### Comparison v

comparison.v.jags= data.frame(
  Group = 1:ncol(post.v.jags),
  Mean.JAGS = colMeans(post.v.jags),
  SD.JAGS   = apply(post.v.jags, 2, sd),
  Mean.JAGS2   = colMeans(post.v.jags2),
  SD.JAGS2     = apply(post.v.jags2, 2, sd)
)
kable(comparison.v.jags, format = "markdown", caption = "Comparison between the v parameters in R and JAGS")


###Visualisation of the differences in the parameters

##Beta
dif.beta.jags = rbind(
  data.frame(value = as.vector(post.beta.jags),
             parameter = rep(c("beta0", "beta1", "beta2"), each = nrow(post.beta.jags)),
             model = "JAGS"),
  data.frame(value = as.vector(post.beta.jags2),
             parameter = rep(c("beta0", "beta1", "beta2"), each = nrow(post.beta.jags2)),
             model = "JAGS2")
)

ggplot(dif.beta.jags, aes(x = value, fill = model)) +
  geom_density(alpha = 0.5) +
  facet_wrap(~parameter, scales = "free") +
  theme_minimal() +
  labs(title = "Posterior distributions of beta parameters",
       x = "Value", y = "Density")


##Vg
```

```r

#We add the confidence intervals to our data.framme of differences
comparison.v.jags$Lower.JAGS = comparison.v.jags$Mean.JAGS - 1.96 * comparison.v.jags$SD.JAGS
comparison.v.jags$Upper.JAGS = comparison.v.jags$Mean.JAGS + 1.96 * comparison.v.jags$SD.JAGS
comparison.v.jags$Lower.JAGS2 = comparison.v.jags$Mean.JAGS2 - 1.96 * comparison.v.jags$SD.JAGS2
comparison.v.jags$Upper.JAGS2 = comparison.v.jags$Mean.JAGS2 + 1.96 * comparison.v.jags$SD.JAGS2


v.long = bind_rows(
  comparison.v.jags %>%
    mutate(Group = 1:n()) %>%
    transmute(
      Group,
      Model = "JAGS",
      Mean = Mean.JAGS,
      Lower = Mean.JAGS - 1.96 * SD.JAGS,
      Upper = Mean.JAGS + 1.96 * SD.JAGS
    ),
  comparison.v.jags %>%
    mutate(Group = 1:n()) %>%
    transmute(
      Group,
      Model = "JAGS2",
      Mean = Mean.JAGS2,
      Lower = Mean.JAGS2 - 1.96 * SD.JAGS2,
      Upper = Mean.JAGS2 + 1.96 * SD.JAGS2
    )
)

ggplot(v.long, aes(x = Group, y = Mean, color = Model)) +
  geom_point(position = position_dodge(width = 0.5)) +
  geom_errorbar(aes(ymin = Lower, ymax = Upper),
                width = 0.2,
                position = position_dodge(width = 0.5)) +
  theme_minimal() +
  labs(title = "Posterior␣Means␣and␣95%␣CIs␣for␣v.g␣by␣Model",
       x = "Group␣(hospital)",
       y = "Mean␣  ␣95%␣CI")
```